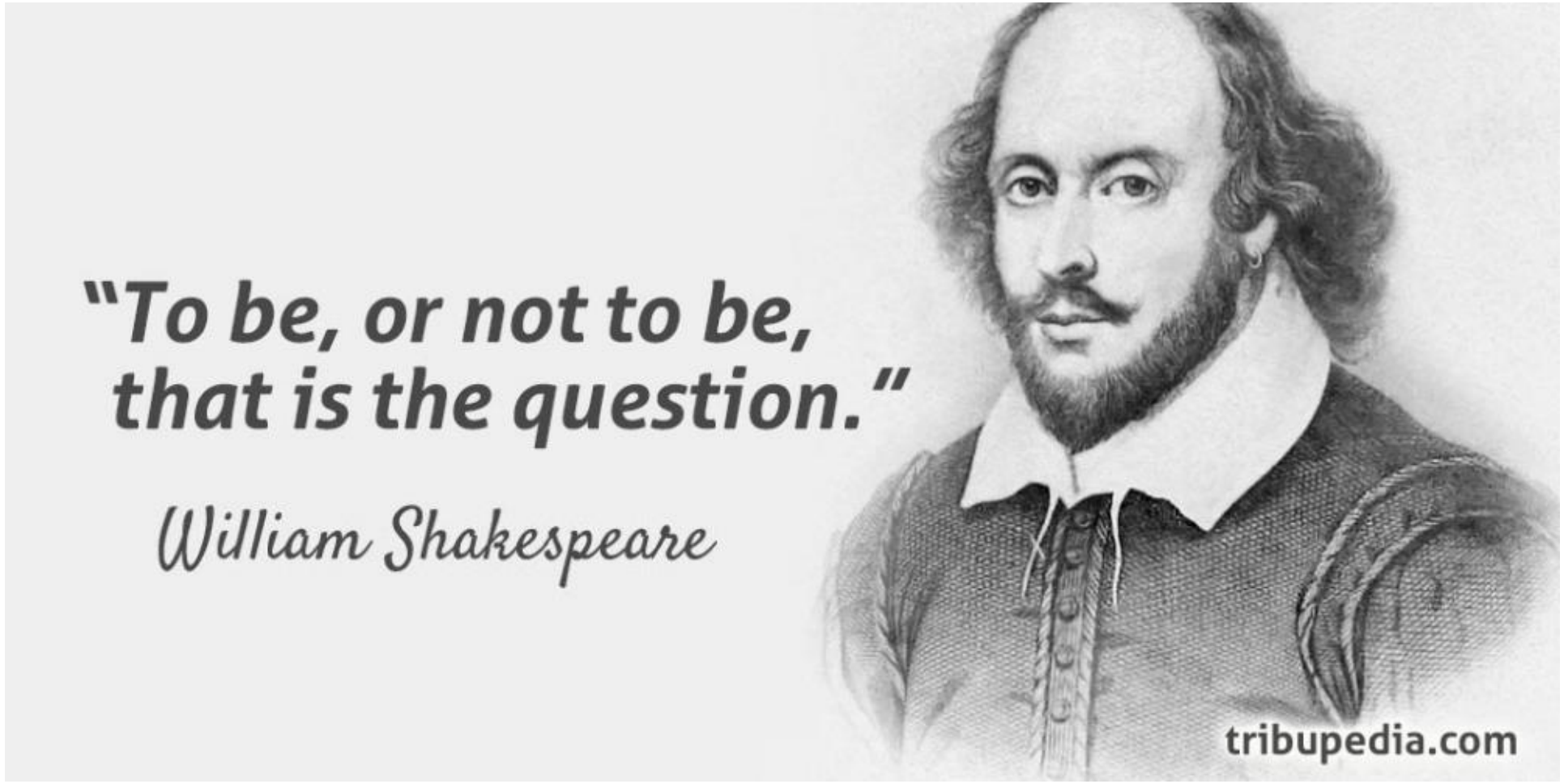


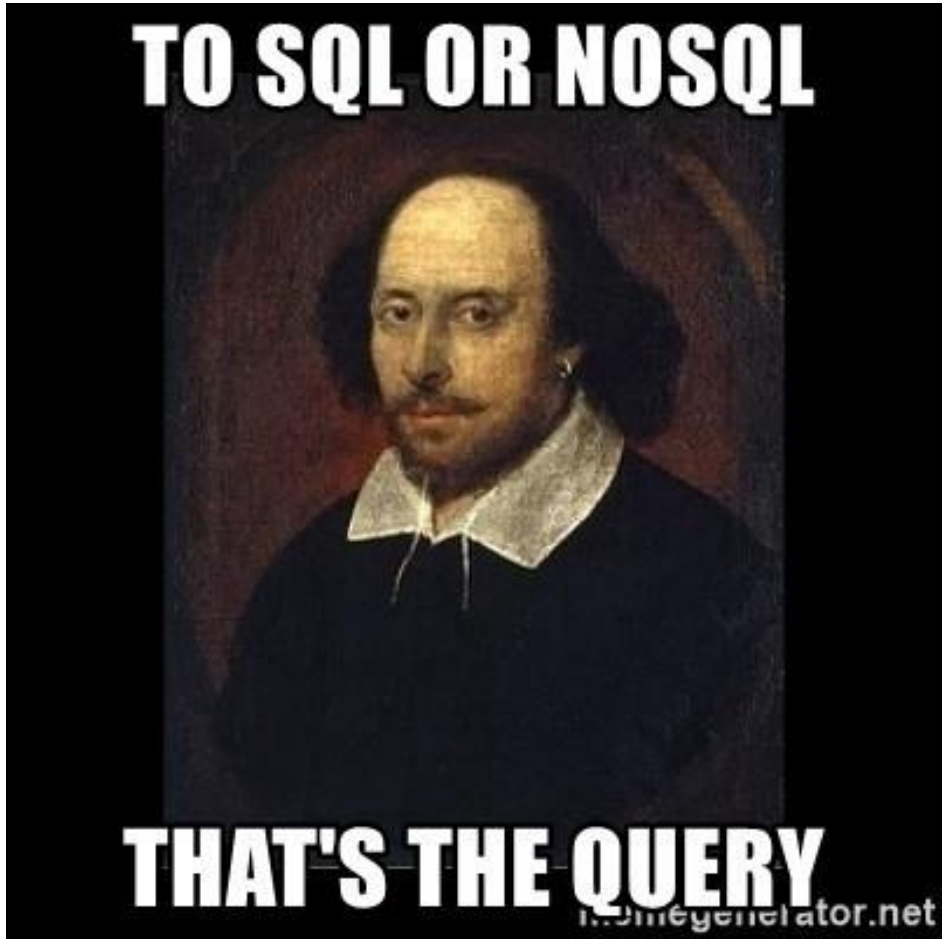
Big Data

Chapter 4

Characteristics of NoSQL Databases

Jorge Bernardino
jorge@isec.pt


"To be, or not to be,
that is the question."
William Shakespeare
tribupedia.com


TO SQL OR NOSQL
THAT'S THE QUERY

Agenda

- NoSQL Architectures
- CAP Theorem
- Database Sharding
- Sharding Architectures
- ACID and BASE models
- Advantages of NoSQL
- Principal Features of NoSQL

NoSQL Architectures

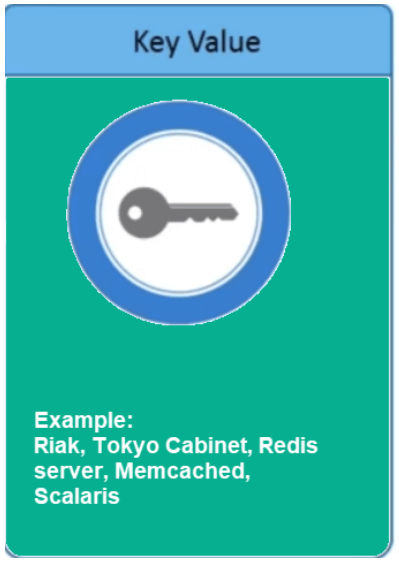
NoSQL Databases are mainly categorized into four types:

- Key-value
- Column-oriented
- Document-oriented
- Graph-based

NoSQL Architectures

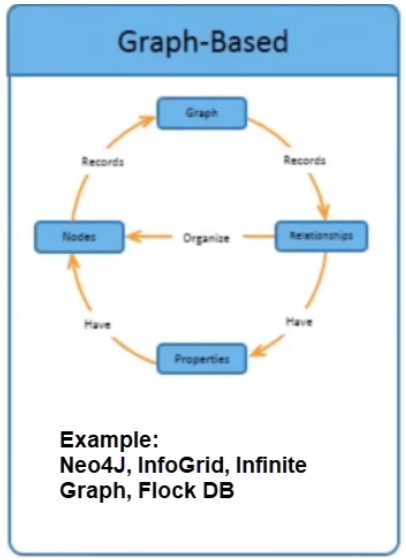
- Every category has its unique attributes and limitations.
- None of the above-specified database is better to solve all the problems.
- Users should select the database based on their needs.

Types of NoSQL Databases





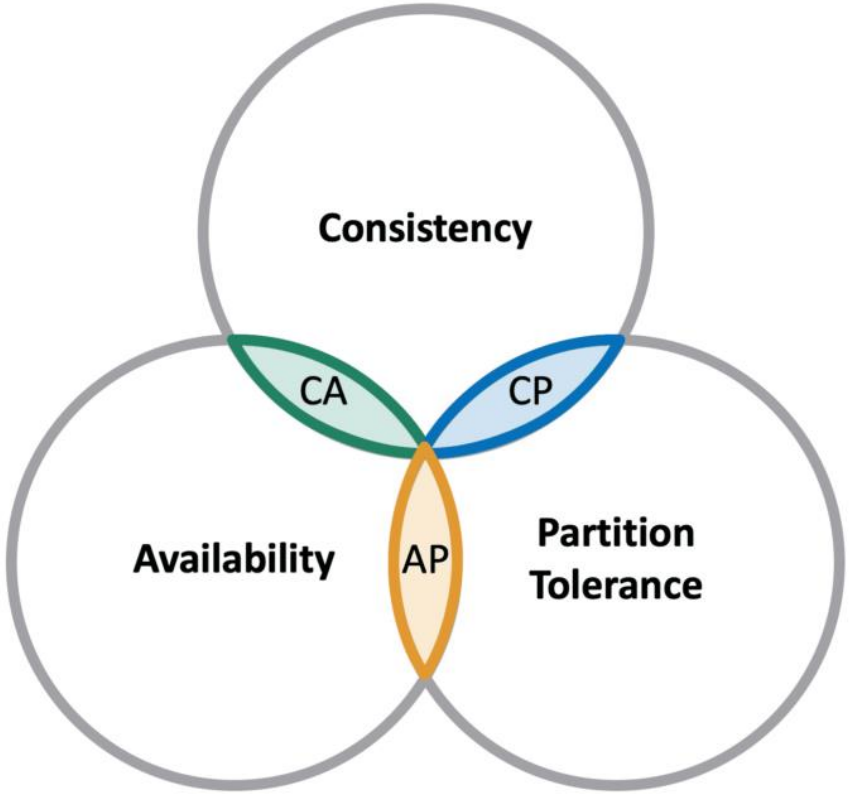




What is the CAP Theorem?

- CAP theorem is also called Brewer's theorem.
- It states that is impossible for a distributed database to offer more than two out of three guarantees:
  - Consistency
  - Availability
  - Partition Tolerance

What is the CAP Theorem?



CAP Theorem

Consistency

- The data should remain consistent even after the execution of an operation.
- This means once data is written, any future read request should contain that data.
- For example, after updating the order status, all the clients should be able to see the same data.

CAP Theorem

Availability

- The database should always be available and responsive.
- It should not have any downtime.

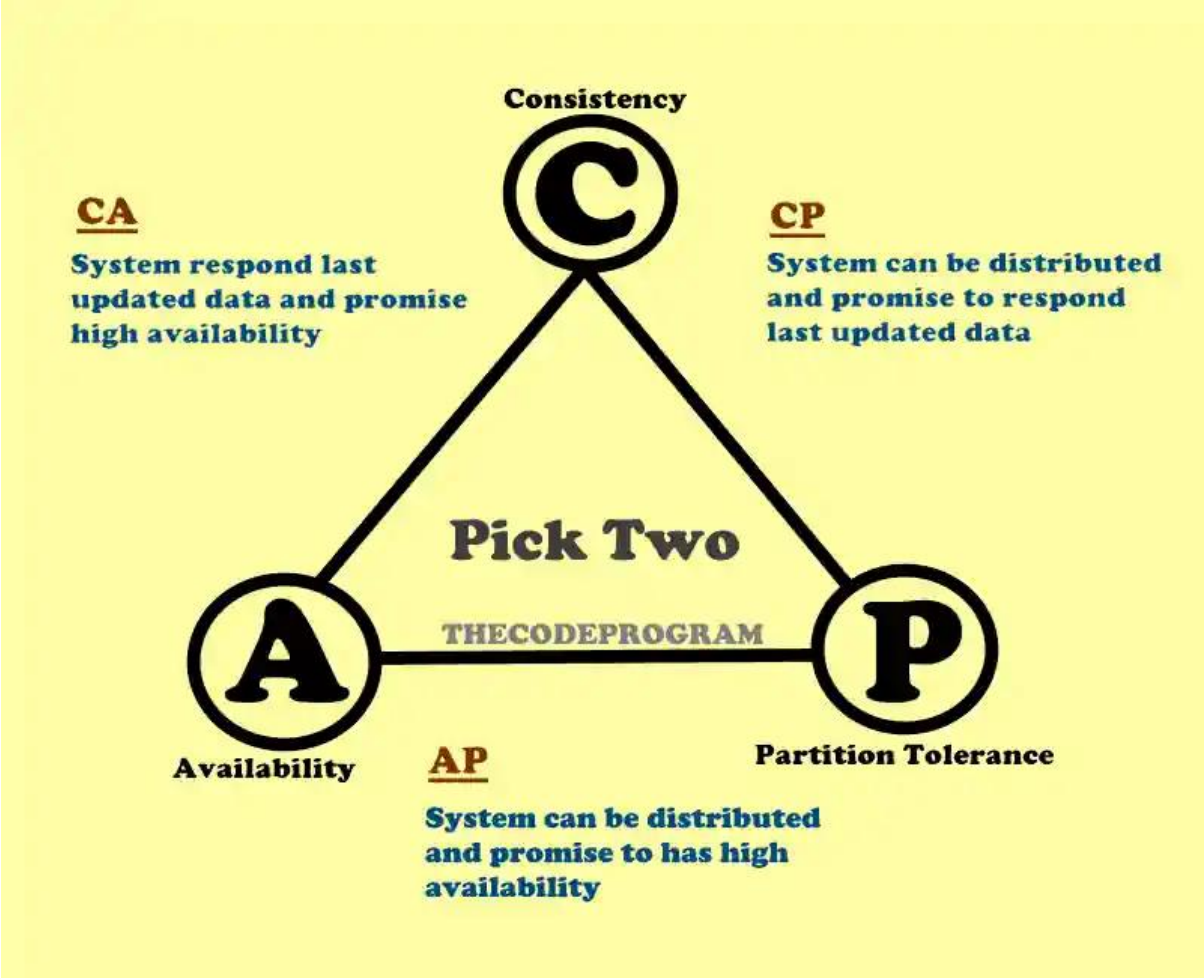


CAP Theorem

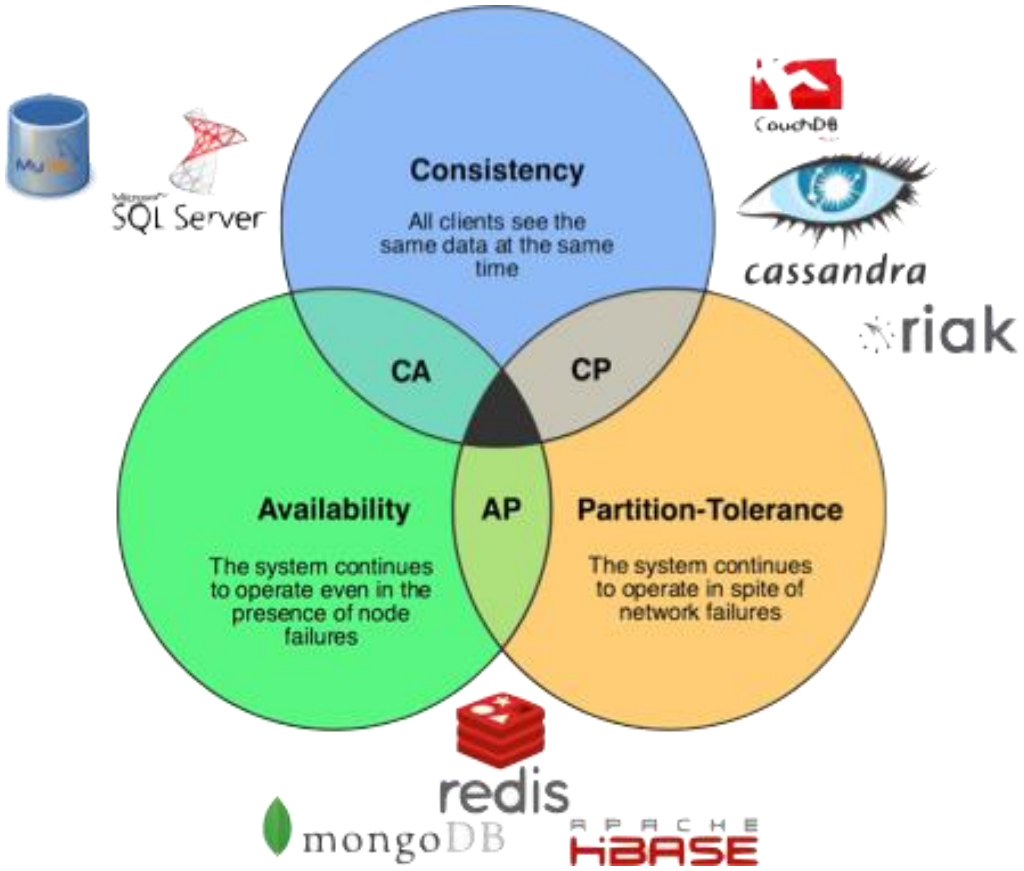
Partition Tolerance

- Partition Tolerance means that the system should continue to function even if the communication among the servers is not stable.
- For example, the servers can be partitioned into multiple groups which may not communicate with each other. Here, if part of the database is unavailable, other parts are always unaffected.

CAP Theorem


Consistency
C
CA
System respond last updated data and promise high availability
CP
System can be distributed and promise to respond last updated data
Pick Two
THECODEPROGRAM
A
P
Availability
Partition Tolerance
AP
System can be distributed and promise to has high availability

CAP Theorem



Lourenço, J.R., Cabral, B., Carreiro, P. ,Vieira, M., Bernardino, J. "Choosing the right NoSQL database for the job: a quality attribute evaluation". *Journal of Big Data* **2,** 18 (2015). https://journalofbigdata.springeropen.com/articles/10.1186/s40537-015-0025-0

Database Sharding

- Sharding is a database architecture pattern related to horizontal partitioning - the practice of separating one table's rows into multiple different tables, known as partitions.
- Each partition has the same schema and columns, but also entirely different rows.
- Likewise, the data held in each is unique and independent of the data held in other partitions.

Database Sharding (Google Bard)

Explain me what is database sharding. If possible with some pictures

- Database Sharding is a technique used to distribute data horizontally across multiple physical databases, or "shards," to improve performance and scalability.
- Imagine you have a massive library with millions of books. Instead of storing all the books in one giant room, you could divide them up and store them in multiple smaller rooms (shards).
- This makes it easier and faster to find the specific book you're looking for.

Database Sharding (Gemini)

Explain me what is database sharding. If possible with some pictures

- Database Sharding is a method used to scale databases across multiple servers or instances. It involves splitting a large database into smaller, more manageable pieces called "shards."
- Each shard contains a portion of the data, allowing for improved performance, scalability, and easier management.

Database Sharding (Google Bard)

How it works ?

- Data Partitioning: The first step is to divide your data into smaller, independent chunks based on a chosen key, like user ID, location, or date. This key determines which shard a particular piece of data belongs to.
- Routing Requests: When an application needs to access data, it sends a request to a "shard router." This router acts like a librarian who knows where each book is located. It identifies the appropriate shard based on the data key and directs the request to the correct database server.
- Independent Operations: Each shard operates independently, meaning you can add or remove servers without affecting the others. This allows you to scale your database horizontally by simply adding more shards as your data grows.

Database Sharding (Google Bard)

Benefits of Database Sharding:

- Improved Performance: Distributing data across multiple servers reduces the load on each individual server, leading to faster read and write operations.
- Enhanced Scalability: You can easily scale your database by adding more shards as your data volume increases.
- High Availability: If one shard fails, the others can still function, ensuring uninterrupted access to your data.

Challenges of Database Sharding:

- Increased Complexity: Managing multiple databases and routing requests can be more complex than managing a single database.
- Data Consistency: Maintaining data consistency across all shards can be challenging, especially for frequently updated data.
- Joins and Aggregations: Performing joins and aggregations across multiple shards can be complex and computationally expensive..

Database Sharding (Google Bard) - References

Give me the references for this Database Sharding definition

Books:

- Database Internals: A Guide to Relational Database Architecture and Optimization by Alex Petrov (2019). This book dives deep into database architecture and includes a chapter on horizontal scaling and sharding 😀.
- Designing Data-Intensive Applications by Martin Kleppmann (2017). This comprehensive book covers various data management techniques, including a detailed section on sharding and its considerations 😀.

Blog Posts and Articles:

- What is Database Sharding? by Amazon Web Services: https://aws.amazon.com/blogs/database/sharding-with-amazon-relational-database-service/ 😕
- Database Sharding Explained: Concepts & Examples by MongoDB: https://www.mongodb.com/docs/manual/sharding/ 😕
- Database Sharding Defined: Concepts and Tips for Success by PingCAP: https://docs.pingcap.com/tidb/stable/shard-diff/ 😕

Research Papers:

- A Survey of Scalable Data Management Systems by Michael Stonebraker et al. (2005): https://apps.dtic.mil/sti/pdfs/AD1034695.pdf 😔
- Big Data: Principles and Best Practices by Jimmy Lin and Chris Dyer (2014): https://www.oreilly.com/library/view/big-data/9781617290343/ 😔

Database Sharding (Gemini)

Key Concepts of Database Sharding

- **Horizontal Partitioning**: The main idea behind sharding is to horizontally partition data. Instead of storing all records in a single database instance, data is split across several instances or servers.
- **Shards**: Each shard operates as an independent database that contains a subset of the overall data. For example, if you have a user database, you might shard it by geographical location (e.g., users from North America on one server, users from Europe on another).
- **Load Balancing**: By distributing data across multiple shards, you can balance the load between different servers. This leads to faster query performance and reduced resource contention.
- **Scalability**: Sharding allows for easier scaling since new shards can be added as needed to accommodate growing data sizes or increasing operational demands.
- **Data Distribution**: Different strategies can be employed to distribute data across shards:

  Range-based sharding: Ranges of data to specific shards (e.g., user IDs 1-1000 on shard 1, 1001-2000 shard 2).

  Hash-based sharding: Using a hashing function to distribute data more randomly across shards.

Database Sharding (Gemini)

Benefits of Database Sharding

- **Performance**: Queries can be processed faster as they only target a specific shard.
- **Scalability**: New shards can be added to handle more data and traffic.
- **Fault Isolation**: Issues in one shard don't necessarily affect others, enhancing fault tolerance.

Database Sharding (OpenAI GPT 5.2) IAEdu

Explain me what is database sharding. If possible with some pictures

What is Database Sharding?

- **Database Sharding** is a way to **split one logical database into multiple smaller databases ("shards")**, usually to **scale horizontally** (handle more data and more traffic by adding machines).
- Each shard holds **only a subset of the rows** (data), and together the shards make up the full dataset.

Database Sharding (OpenAI GPT 5.2) IAEdu

What sharding looks like?

1) Before: one big database

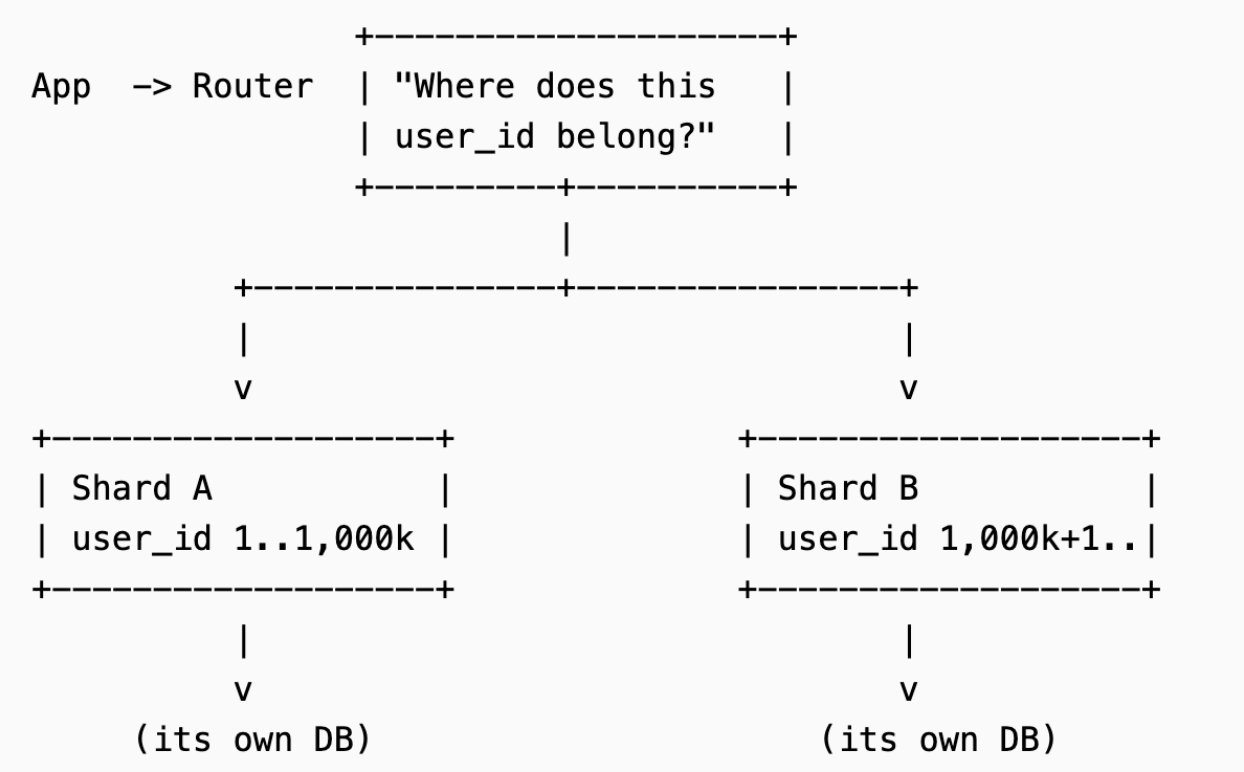
```
App
 |
 v
+-----------------+
| One Database |
| (all users/data) |
+-----------------+
```

Database Sharding (OpenAI GPT 5.2) IAEdu

What sharding looks like?

2) After: data split across shards

Example: split users by user_id ranges.



Database Sharding (OpenAI GPT 5.2) IAEdu

How data is assigned to a shard (common strategies)

A) Range-based sharding

- Rule: user_id 1..1,000,000 -> Shard A, 1,000,001..2,000,000 -> Shard B
- Pros: simple, good for range queries
- Cons: "hot spots" if most activity is in one range

B) Hash-based sharding

- Rule: shard = hash(user_id) % N
- Pros: tends to distribute load evenly
- Cons: range queries become harder; changing N can be painful unless designed for it

C) Directory / lookup-based sharding

- Store a mapping: user_id -> shard
- Pros: flexible, can move specific tenants/users
- Cons: that directory becomes critical infrastructure

Database Sharding (OpenAI GPT 5.2) IAEdu

Why people shard?

- **Scale storage**: each shard stores less data
- **Scale throughput**: reads/writes spread across shards
- **Isolation**: a busy shard doesn't necessarily take down everything

Trade-offs (important)

- **Cross-shard queries** (joins, aggregates) get harder/slower
- **Transactions across shards** are more complex
- **Rebalancing** (moving data when shards get full/uneven) is operationally tricky
- More components to monitor and maintain.

Database Sharding (Chat GPT)

How Does Sharding Work?

- Instead of storing all data in a single database, sharding distributes the data across multiple servers (shards). Each shard typically contains a unique subset of the data, often based on a key such as user ID, geographic region, or timestamp.

Example of Sharding:

Imagine a social media application with millions of users. Instead of storing all users in one database, the data is split as follows:

- Shard 1: Users with IDs from 1 to 1 million
- Shard 2: Users with IDs from 1 million to 2 million
- Shard 3: Users with IDs above 2 million

Each shard operates independently, reducing the load on a single database.

Database Sharding (Chat GPT)

Benefits of Database Sharding:

✅ Scalability – Databases can grow without performance degradation.

✅ Performance Boost – Queries run faster since each shard handles less data.

✅ Fault Isolation – If one shard fails, the rest remain operational.

✅ Easier Maintenance – Smaller databases are easier to manage

Challenges of Database Sharding:

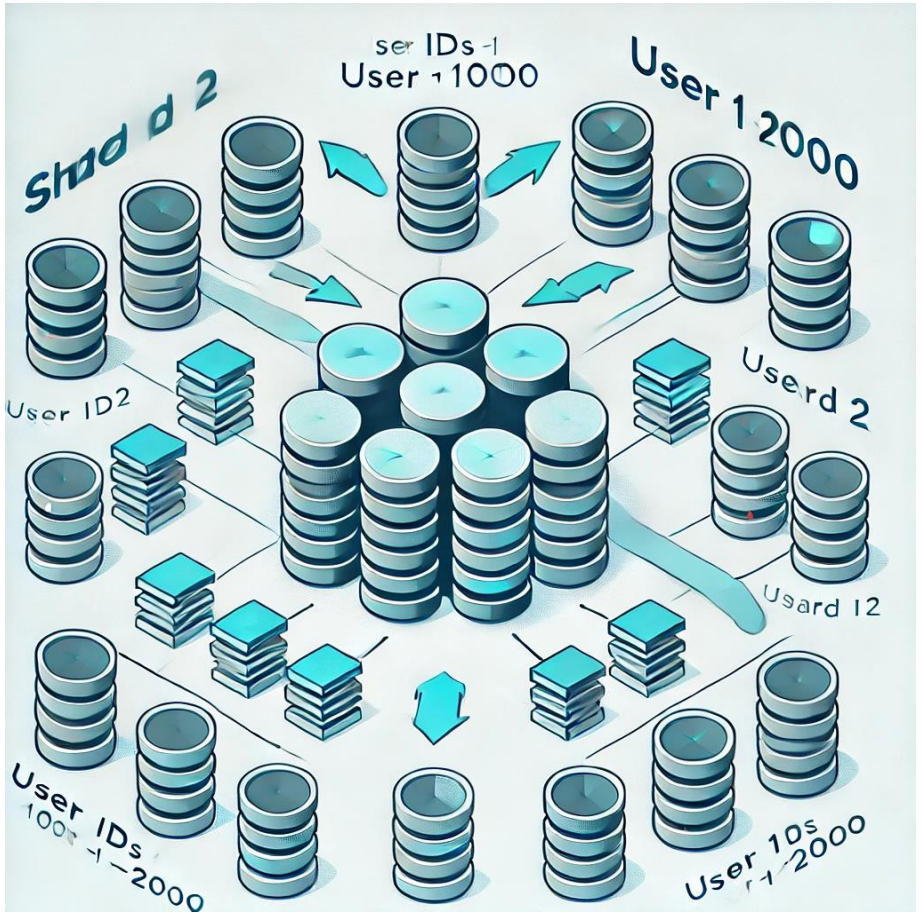
⚠️ Complex Implementation – Requires careful planning and design.

⚠️ Cross-Shard Queries – Queries across multiple shards can be slow and complex.

⚠️ Rebalancing Data – When shards grow unevenly, rebalancing is needed.

Database Sharding (Chat GPT)

Visual Representation of Sharding:



Database Sharding (Chat GPT) - References

Give me the references for this Database Sharding definition

Certainly! Here are some authoritative references that provide detailed definitions and explanations of database sharding:

- Amazon Web Services (AWS): AWS defines database sharding as a horizontal scaling strategy that distributes data across multiple nodes or computers to share the workload of an application. This approach enhances performance and fault tolerance. AWS.AMAZON.COM
- Microsoft Azure: According to Azure, database sharding involves splitting large databases into smaller, more manageable components through horizontal partitioning. Each shard exists on a separate database server instance, effectively distributing the load. AZURE.MICROSOFT.COM
- MongoDB: MongoDB describes sharding as a method for distributing a single dataset across multiple databases, which can then be stored on multiple machines. This allows for larger datasets to be split into smaller chunks and stored in multiple data nodes, increasing the total storage capacity of the system. MONGODB.COM
- DigitalOcean: DigitalOcean explains that sharding involves breaking up one's data into two or more smaller chunks, called logical shards. These logical shards are then distributed across separate database nodes, referred to as physical shards, collectively representing the entire dataset. DIGITALOCEAN.COM
- Wikipedia: The Wikipedia entry on database sharding defines it as a horizontal partition of data in a database or search engine. Each shard is held on a separate database server instance to spread the load, with some data present in all shards and some appearing only in a single shard. EN.WIKIPEDIA.ORG

These sources offer comprehensive insights into the concept, implementation, and benefits of database sharding.

Chat GPT atual

🔀 What Is Sharding?

Sharding is a database scaling technique where data is split across multiple machines (called *shards*) so that each machine stores only part of the total dataset.

Instead of one large database handling everything, sharding distributes the load horizontally.

🧩 Simple Analogy

Imagine a library:

- ❌ One giant room with **all books** → gets crowded and slow
- ✅ Multiple rooms, each holding books by **alphabet range (A–F, G–M, N–Z)** → faster access, less congestion

Each room = a **shard**

🏗️ How Sharding Works

Data is divided using a **shard key** — a field that determines where data lives.

For example:

- Users with IDs 1–1,000 → Shard A
- IDs 1,001–2,000 → Shard B
- IDs 2,001–3,000 → Shard C

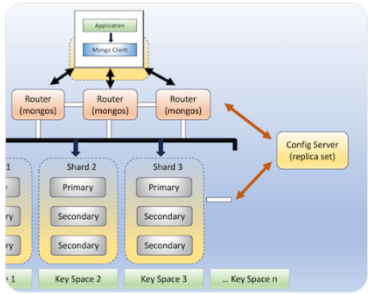
Or:

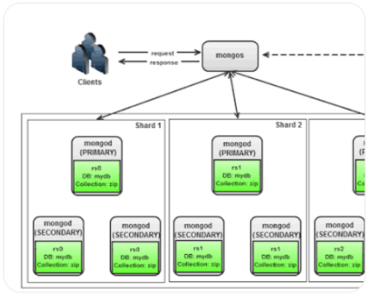
- Europe users → EU shard
- US users → US shard

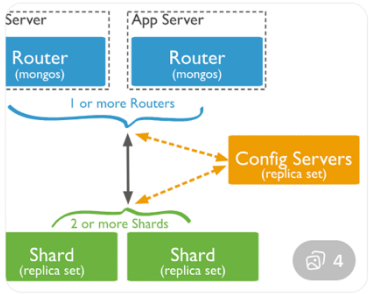
Chat GPT atual

Sharding in Databases

Example: MongoDB



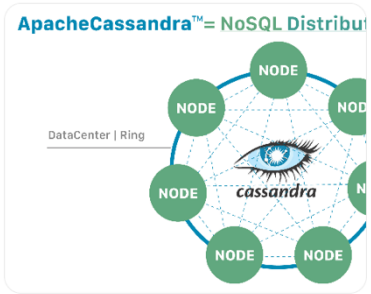


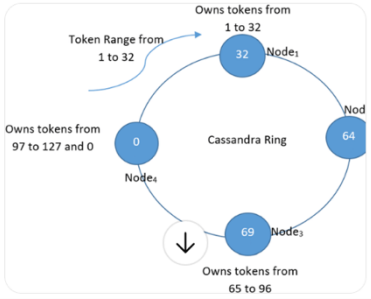


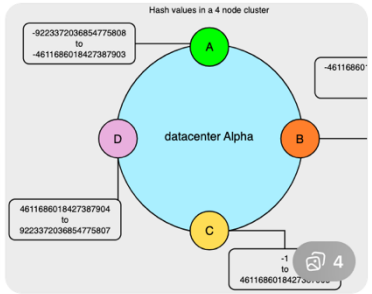
MongoDB supports **automatic sharding**:

- Data is split into chunks
- A router directs queries to the correct shard
- Config servers track metadata

Example: Apache Cassandra







Chat GPT atual

📈 Why Use Sharding?

✅ Benefits

- Horizontal scalability
- Better performance under high traffic
- Handles massive datasets
- Fault isolation (one shard fails ≠ whole system fails)

⚠️ Challenges

- Complex architecture
- Harder joins across shards
- Data rebalancing issues
- Requires careful shard key design

🔍 Sharding vs Replication

Sharding	Replication
Splits data	Copies data
Scales storage	Improves availability
Different data per node	Same data per node

Often, systems use **both together**.

Chat GPT atual - References

Foundational & Classic Academic Papers

1. Bagui & Nguyen – Database Sharding (2015), *Sikha Bagui & Loi Nguyen.*
"Database Sharding", *International Journal of Cloud Applications and Computing*, 2015.
A comprehensive academic treatment of sharding as horizontal partitioning for highly scalable distributed databases. (ResearchGate)
Why it's useful: widely cited in distributed database research, focuses on architecture, scalability, high availability and fault tolerance using sharding.

2. Hohenstein & Jaeger – Economical Aspects of Database Sharding (2014), *Uwe Hohenstein & Michael C. Jaeger.*
"Economical Aspects of Database Sharding", SCITEPRESS, 2014.
Explores sharding from cost optimization and cloud scalability perspectives. (SciTePress)
Why it's useful: useful if you want to understand **trade-offs in cloud environments** and **real deployment cost considerations**.

3. Costa et al. – Sharding by Hash Partitioning (2015), *Caio H. Costa, João Vianney B. M. Filho, et al.*
"Sharding by Hash Partitioning – A Database Scalability Pattern…", SCITEPRESS, 2015.
Discusses **hash-based sharding strategies** and their effects on load distribution and performance. (SciTePress)
Why it's useful: excellent reference for **data distribution methods** in sharded systems.

Chat GPT atual - References

Recent & Advanced Research

4. Solat – Sharding Distributed Databases: A Critical Review (2024), *Siamak Solat.*
"Sharding Distributed Databases: A Critical Review", arXiv 2024.
A recent survey examining challenges and performance limitations of sharded distributed replication systems. (ResearchGate)
Why it's useful: up-to-date overview of implementation challenges and comparisons across distributed systems.

5. Thakur et al. – Self-Healing Nodes with Adaptive Data-Sharding (2024), *Ayush Thakur et al.*
"Self-healing Nodes with Adaptive Data-Sharding", arXiv 2024.
Proposes adaptive and dynamic sharding with load balancing and fault tolerance mechanisms. (arXiv)
Why it's useful: research that pushes beyond static sharding, relevant for next-generation self-managing distributed databases.

6. Hong et al. – GriDB: Scaling Blockchain Database via Sharding (2024), *Zicong Hong et al.*
"GriDB: Scaling Blockchain Database via Sharding…", arXiv 2024.
Although applied to blockchain, this paper investigates **cross-shard services and scalable query mechanisms** in distributed data environments. (arXiv)
Why it's useful: highlights sharding challenges in decentralized systems, with advanced consensus and inter-shard design.

Chat GPT atual - References

📘 Additional Related Studies

Sharding for Scalable Blockchain Networks – Review of sharding protocols and scalability issues in distributed ledger systems. (UAE Ministry of Health)

Efficient Data Migration Strategies in Sharded Databases – paper covering migration strategies and data movement between shards. (jqst.mindsynk.org)

📌 How to Access These Papers

Most of the above papers are available through:

arXiv (open access) — especially for recent work like adaptive sharding and blockchain sharding.

SCITEPRESS.org — conference publications on hash partitioning and economic aspects.

ResearchGate — often hosts postprints of journal articles like the *Database Sharding* paper by Bagui & Nguyen.

If you want, I can prepare **proper APA or BibTeX citations** for these or help you find **PDFs and DOI links** where available.

Database Sharding

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Vertical Partitions

VP1

CUSTOMER ID	FIRST NAME	LAST NAME
1	TAEKO	OHNUKI
2	O.V.	WRIGHT
3	SELDA	BAĞCAN
4	JIM	PEPPER

VP2

CUSTOMER ID	FAVORITE COLOR
1	BLUE
2	GREEN
3	PURPLE
4	AUBERGINE

Horizontal Partitions

HP1

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN

HP2

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Database Sharding

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Vertical Partitions

VP1

CUSTOMER ID	FIRST NAME	LAST NAME
1	TAEKO	OHNUKI
2	O.V.	WRIGHT
3	SELDA	BAĞCAN
4	JIM	PEPPER

VP2

CUSTOMER ID	FAVORITE COLOR
1	BLUE
2	GREEN
3	PURPLE
4	AUBERGINE

Horizontal Partitions

HP1

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN

HP2

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Sharding

- Sharding involves breaking up one's data into two or more smaller chunks, called *logical shards*.
- The logical shards are then distributed across separate database nodes, referred to as *physical shards*, which can hold multiple logical shards.
- Despite this, the data held within all the shards collectively represent an entire logical dataset.

Sharding

- Database shards exemplify a shared-nothing architecture. This means that the shards are autonomous; they don't share any of the same data or computing resources.
- In some cases, though, it may make sense to replicate certain tables into each shard to serve as reference tables.
- For example, let's say there's a database for an application that depends on fixed conversion rates for weight measurements.
- By replicating a table containing the necessary conversion rate data into each shard, it would help to ensure that all of the data required for queries is held in every shard.

Sharding Architectures

Key Based Sharding

- Key based sharding, also known as hash based sharding, involves using a value taken from newly written data — such as a customer's ID number, a client application's IP address, a ZIP code, etc. — and plugging it into a hash function to determine which shard the data should go to.
- A hash function is a function that takes as input a piece of data (for example, a customer email) and outputs a discrete value, known as a hash value.
- In the case of sharding, the hash value is a shard ID used to determine which shard the incoming data will be stored on.

Sharding Architectures

Key Based Sharding

Example of a hash function

- Let's consider a simple hash function that processes individual characters and assigns them numerical values (A=1, B=2, etc.) and then adds them up.

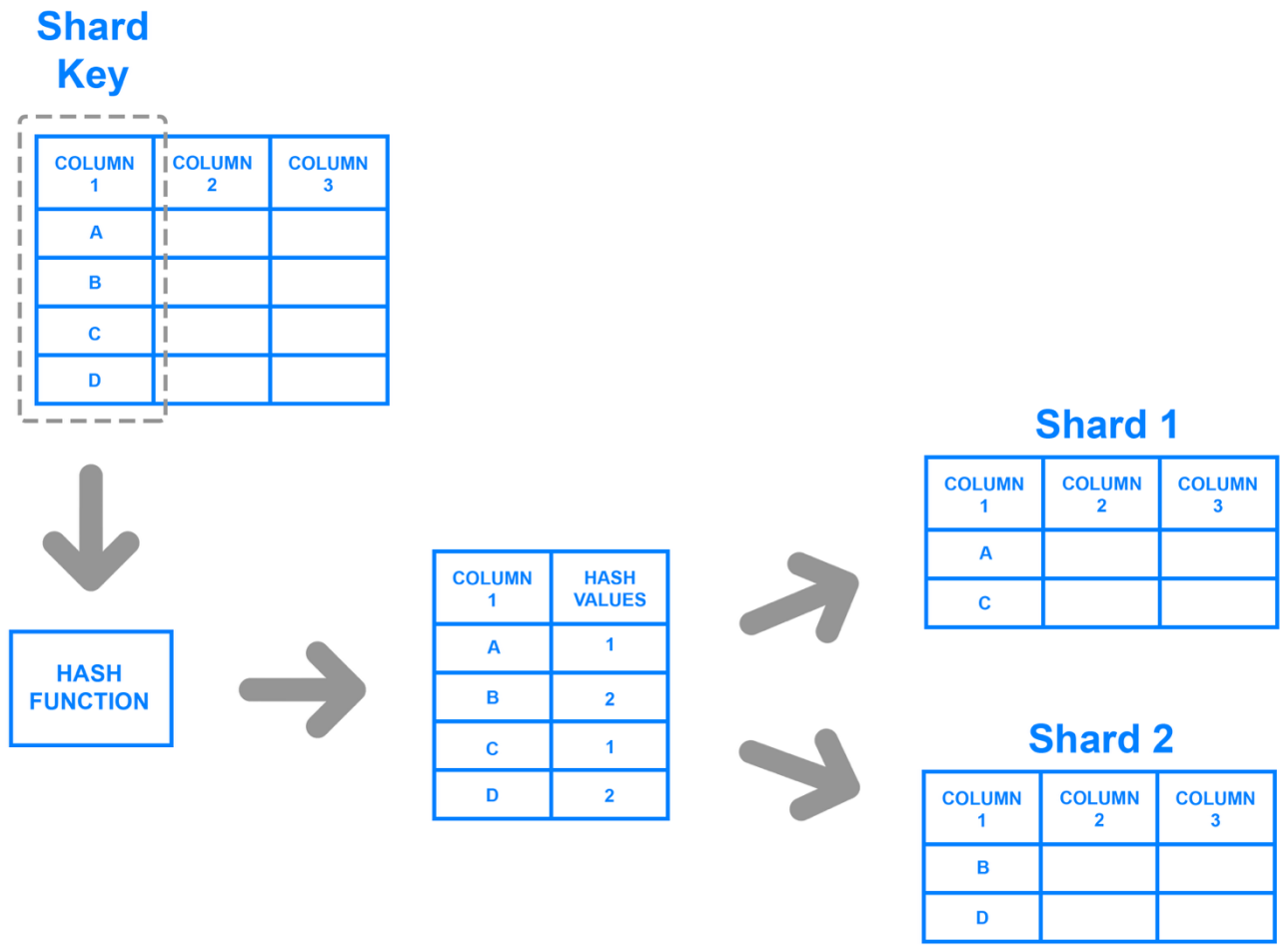
  Input: "HELLO"

  Processing: H (8) + E (5) + L (12) + L (12) + O (15) = 52

  Hash value: 52

Even though "HELLO" is a short string, its hash value is unique and different from other words like "Hi" (18) or "World" (77).

Key Based Sharding



Sharding Architectures

Range Based Sharding

- *Range based sharding* involves sharding data based on ranges of a given value.
- If we have a database that stores information on all the products in a retailer's catalogue, we can create a few different shards and divide up the information on each product based on the price range it falls into.
- Example:

Range Based Sharding

PRODUCT	PRICE
WIDGET	$118
GIZMO	$88
TRINKET	$37
THINGAMAJIG	$18
DOODAD	$60
TCHOTCHKE	$999



($0-$49.99)

TRINKET	$37
THINGAMAJIG	$18

($50-$99.99)

GIZMO	$88
DOODAD	$60

($100+)

WIDGET	$118
TCHOTCHKE	$999

Range Based Sharding

PRODUCT	PRICE
WIDGET	$118
GIZMO	$88
TRINKET	$37
THINGAMAJIG	$18
DOODAD	$60
TCHOTCHKE	$999



($0-$49.99)

PRODUCT	PRICE
TRINKET	$37
THINGAMAJIG	$18

($50-$99.99)

PRODUCT	PRICE
GIZMO	$88
DOODAD	$60

($100+)

PRODUCT	PRICE
WIDGET	$118
TCHOTCHKE	$999

Consistency

- ACID (**A**tomicity, **C**onsistency, **I**solation, and **D**urability)
- BASE (**B**asic **A**vailability, **S**oft state, and **E**ventual consistency)



What is ACID?

- The idea of transactions, their semantics and guarantees, evolved with data management itself.
- As computers became more powerful, they were tasked with managing more data.
- Eventually, multiple users shared data on a machine.
- This led to problems where data could be changed or overwritten while other users were in the middle of a calculation.

What is ACID?



https://www.youtube.com/watch?v=yaQ5YMWkxq4

What is ACID?

A — **Atomicity** — either all of the operations in a transaction succeed, or none of them do

C — **Consistency** — the database enforces rules about its fields and the relationships between fields

I — **Isolation** — the extent to which rows that a transaction is affecting can be affected by other transactions

D — **Durability** — the database writes its data to a permanent medium (hard drive) so that data is not lost during a power failure or other system failure.

What does ACID stand for?

- **Atomic**: All components of a transaction are treated as a single action. All are completed or none are; if one part of a transaction fails, the database's state is unchanged.

- **Consistent**: Transactions must follow the defined rules and restrictions of the database, e.g., constraints, cascades, and triggers. Thus, any data written to the database must be valid and any transaction that completes will change the state of the database. No transaction can create an invalid data state. Note that this is different from "consistency" as it's defined in the CAP theorem.

What does ACID stand for?

- **Isolated**: Fundamental to achieving concurrency control, isolation ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed serially, i.e., one after the other. With isolation, an incomplete transaction cannot affect another incomplete transaction.

- **Durable**: Once a transaction is committed, it will persist and will not be undone to accommodate conflicts with other operations. Many argue that this implies the transaction is on disk as well; most formal definitions aren't specific.

Eventual Consistency

- The term "eventual consistency" means to have copies of data on multiple machines to get high availability and scalability. Thus, changes made to any data item on one machine has to be propagated to other replicas.

- Data replication may not be instantaneous as some copies will be updated immediately while others in due course of time. These copies may be mutually, but in due course of time, they become consistent. Hence, the name eventual consistency.

Eventual Consistency

- **CRUD ?**
- Create/Insert, Read, Update, and Delete (CRUD) are four operations that databases are expected to provide and execute efficiently.
- NoSQL data models are structured to support massively concurrent insert and read operations relative to updates.
- The notion of eventual consistency is embraced to support extremely fast insert and read operations. Eventual consistency implies that applications are aware of the non-repeatable read issue due to latency in consistency.
- Some NoSQL systems employ data models to efficiently support only insert and read to the exclusion of update and delete.

BASE

Basically **A**vailable

Soft State

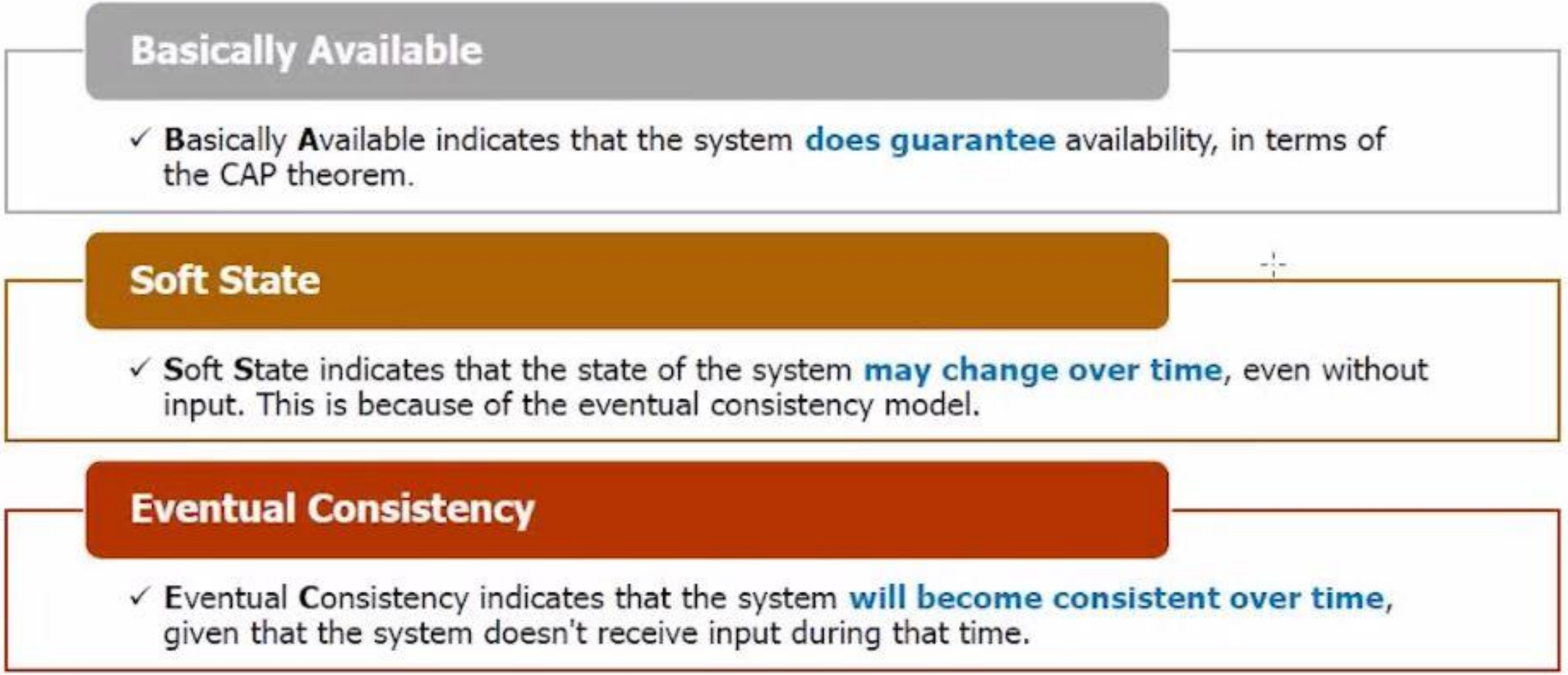
Eventually Consistent

BASE

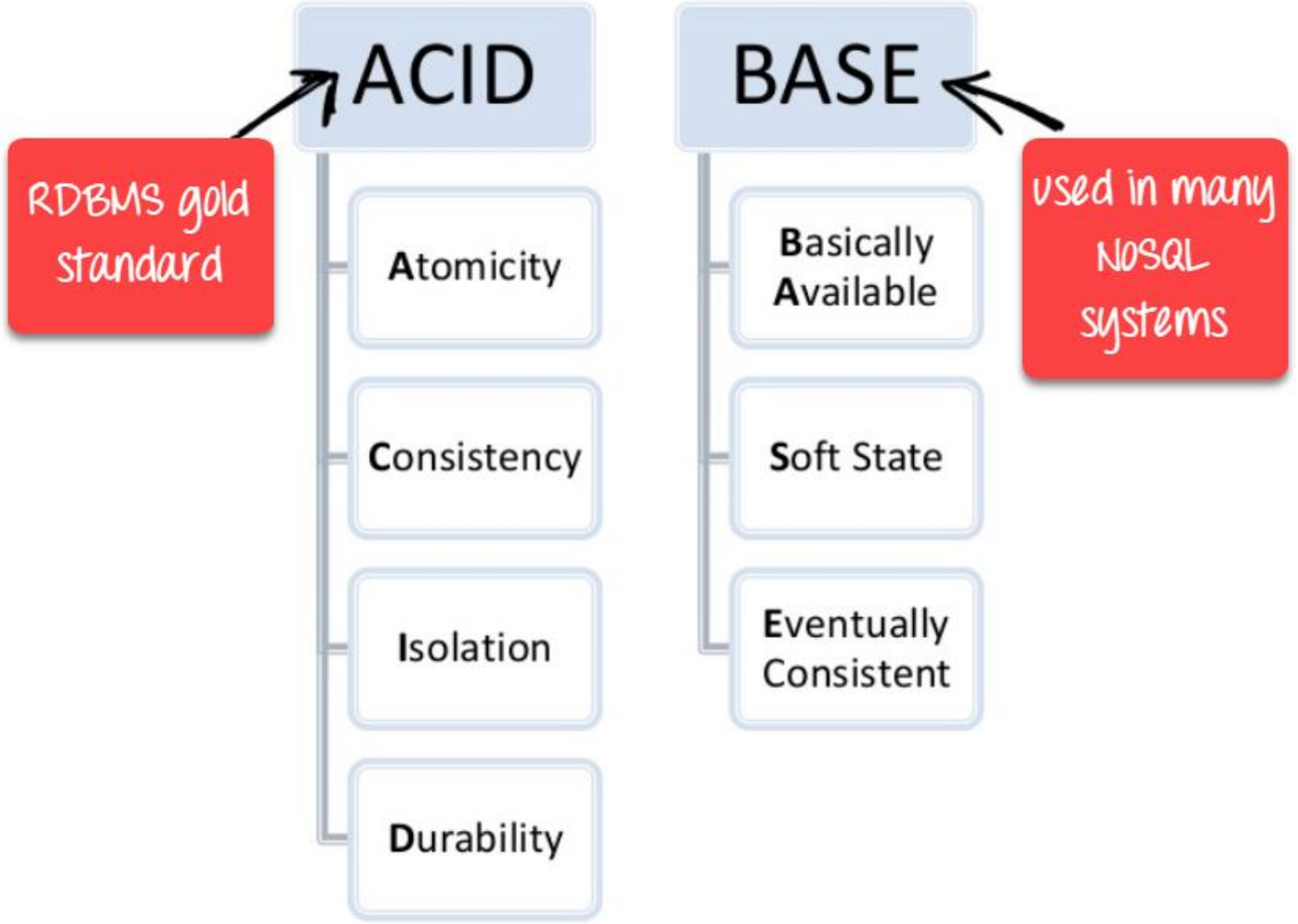
- BASE: **B**asically **A**vailable, **S**oft state, **E**ventual consistency

  - **Basically**, available means DB is available all the time as per CAP theorem
  - **Soft** state means even without an input; the system state may change
  - **Eventual** consistency means that the system will become consistent over time.

BASE


Basically Available
✓ Basically Available indicates that the system does guarantee availability, in terms of the CAP theorem.
Soft State
✓ Soft State indicates that the state of the system may change over time, even without input. This is because of the eventual consistency model.
Eventual Consistency
✓ Eventual Consistency indicates that the system will become consistent over time, given that the system doesn't receive input during that time.

ACID vs BASE


ACID
RDBMS gold standard
Atomicity
Consistency
Isolation
Durability
BASE
used in many NOSQL systems
Basically Available
Soft State
Eventually Consistent

ACID vs BASE

ACID

- **A**tomicity. Only the specified data is affected by an operation.
- **C**onsistency. Each operation moves the database from one consistent state to another consistent state.
- **I**solation. One operation does not affect other concurrent operations.
- **D**urability. The data does not get lost following a successful transaction.

BASE

- **B**asically **A**vailable. Writing and reading operations are available as much as possible but without any strict guarantees.
- **S**oft State. Without guarantees, we do not know but have expectations that the data eventually becomes consistent.
- **E**ventual Consistency. If the system is fully functional, and a long enough period has passed, we will eventually know the true state of the database.

Features of ACID and BASE

ACID	**Atomicity**	**All or nothing of the N actions. Commit or Rollback**
	Consistency	Transactions never observe or cause inconsistent data
	Isolation	Transactions are not aware of concurrent transactions
	Durability	Acknowledged transactions persist in all events
BASE	**Optimistic behavior**	**Accept temporary database inconsistencies**
	Basically Available	Availability by replication
	Soft state	It is the user's application's task to guarantee Consistency
	Eventually consistent	Weakly Consistent, the database will be consistent in the long run; 'stale' data is OK.

Advantages of NoSQL

- Big Data Capability
- No Single Point of Failure
- Easy Replication
- Can be used as Primary or Analytic Data Source
- No Need for Separate Caching Layer
- It provides fast performance and horizontal scalability
- Can handle structured, semi-structured, and unstructured data with equal effect
- Object-oriented programming which is easy to use and flexible.

Advantages of NoSQL (cont.)

- NoSQL databases don't need a dedicated high-performance server
- Support Key Developer Languages and Platforms
- Simple to implement than using RDBMS
- It can serve as the primary data source for online applications
- Handles big data which manages data velocity, variety, volume, and complexity
- Excels at distributed database and multi-data center operations
- Eliminates the need for a specific caching layer to store data
- Offers a flexible schema design which can easily be altered without downtime or service disruption.

Weaknesses of NoSQL

- No standardization rules
- Limited query capabilities
- RDBMS databases and tools are comparatively mature
- It does not offer any traditional database capabilities, like consistency when multiple transactions are performed simultaneously.
- When the volume of data increases it is difficult to maintain unique values as keys become difficult
- Doesn't work as well with relational data
- The learning curve is steep for new developers
- Open source options so not so popular for enterprises.

Principal Features of NoSQL

A. Big Data Characteristics

B. Flexible Data Models

C. Eventual Consistency

D. Partial Record Updates

E. Application Synthesized Relationships

F. Optimized MapReduce Processing

G. Native Support for Versioning

H. New Query Languages

I. Horizontal Scalability

Source: V. N. Gudivada, D. Rao and V.V. Raghavan, "NoSQL Systems for Big Data Management," 2014 IEEE World Congress on Services, Anchorage, AK, USA, 2014, pp. 190-197.

Principal Features of NoSQL

A. Big Data Characteristics

- Data that is too big and complex to capture, store, process, analyze, and interpret using the state-of-the art tools and methods is referred to as Big Data. It is characterized by five Vs: Volume, Velocity, Variety, Value, and Veracity.
- Data size (Volume) is in the order of terabytes or even petabytes and is rapidly heading towards exabytes. Velocity refers to the speed at which the data is generated. Enterprise Big Data is typically heterogeneous (Variety) and is comprised of structured, semi-structured, and unstructured data. Value refers to extracting useful and actionable information from massive data sets. Due to diversity in sources and evolution of data through intermediate processing, issues of security, privacy, trust, and accountability need to be addressed using secure data provenance (Veracity).

Principal Features of NoSQL

B. Flexible Data Models

- All RDBMS databases are based on the relational data model. In contrast, each class of NoSQL system employs a different data model. NoSQL data models inherently support horizontal data partitioning across processors in a cluster or even processors across data centers. RDBMS applications typically work with fixed database schema, though the schema may evolve slowly. Schema change is a well managed activity and the application goes through rigorous software testing.
- In contrast, many NoSQL systems treat schema as a construct that inherently evolves over time. These systems also require schema that allows great variation in row data without incurring the NULL value problems.
- A NoSQL data model closely reflects an individual application need rather than needs of several applications. The data model naturally mirrors the way a subject matter expert (SME) thinks about the data in the domain in the context of a specific application. In other words, a NoSQL application works with a data model that is specifically designed and optimized for that application.

Principal Features of NoSQL

C. Eventual Consistency

- Eventual consistency is a data modeling concept that ensures that updates made to distributed NoSQL databases will eventually be reflected across all nodes.
- This ensures that identical database queries will return the same results after some period of time.
- The notion of eventual consistency is adopted to support extremely fast insert and read operations.
- Eventual consistency implies that applications are aware of the non-repeatable read issue due to latency in consistency.

Principal Features of NoSQL

D. Partial Record Updates

- RDBMS data models are optimized for row-wise processing based on the assumption that applications process most of an entity's attributes.
- In contrast, some NoSQL data models are designed to efficiently perform column-wise processing to support computing aggregates on one or two attributes across all entities.
- For example, data models of column-oriented NoSQL databases resemble a data warehouse star schema where the fact table at the center stores de-normalized row data and each dimension table stores all columns of a column-family.

Principal Features of NoSQL

E. Application Synthesized Relationships

- RDBMS data models are designed to capture one-to-one, one-to-many, and many-to-many relationships between entities.
- In contrast, many NoSQL data models do not explicitly and inherently model relationships (graph data model based NoSQL systems are an exception).
- Most NoSQL systems do not support relational join operation. NoSQL applications are expected to synthesize relationships by processing the data.
- If the data is inherently rich in relationships, many NoSQL data models are a poor fit.

Principal Features of NoSQL

F. Optimized MapReduce Processing

- In RDBMS applications, employing MapReduce processing requires moving the data from the RDBMS to another memory address space within the same machine or to a different one.
- In contrast, some NoSQL systems feature MapReduce as a native functionality to obviate the data movement.

Principal Features of NoSQL

G. Native Support for Versioning

- Mainstream, off-the-shelf RDBMS do not provide support for data versioning. However, an RDBMS data model can be designed to support versioning of an attribute by including two additional attributes (begin date and end date). This is a convoluted approach with strong performance penalties.
- In contrast, some NoSQL data models are designed to provide native support for versioning.

Principal Features of NoSQL

H. New Query Languages

- RDBMS SQL queries in practice tend to be complex, involve joins across multiple tables, consume substantial database resources to execute, and are expensive to write, debug, and maintain. In contrast, many NoSQL systems avoid SQL altogether. Some provide a SQL-like query language such as XQuery, while others require writing programs for querying the database.
- NoSQL databases take a multi-faceted approach to querying including allowing only simple queries through interfaces such as REST API, caching data in main memory, or using database stored procedures as queries.
- For example, in document-oriented NoSQL systems, a database query is a JSON document (technically, a JavaScript expression) specified across all the documents in the collection.
- For graph model based NoSQL systems, queries involve traversing graph structures. Interfaces for graph traversal include JavaScript-based interface (JIG), and special query languages such as SPARQL, RDFS++, and Cypher.

Principal Features of NoSQL

I. Horizontal Scalability

- Horizontal scalability refers to scaling by adding new processors complete with their own disks (aka nodes). Closely associated with horizontal scaling is the partitioning of data. Each node/processor contains only a subset of the data. The process of assigning data to each node is referred to as sharding, which is done automatically in some NoSQL systems.
- It is easier to achieve horizontal scalability with NoSQL databases. The complexity involved in enforcing ACID compliance simply does not exist for most NoSQL systems as they are ACID non-compliant by design. Furthermore, only insert and read operations dominate in NoSQL databases as update and delete operations fade into insignificance in terms of volume.
- NoSQL systems also achieve horizontal scalability by delegating two-phase commit required for transaction implementation to applications. For example, MongoDB does not guarantee ACID compliance for concurrent operations. This may infrequently result in undesirable outcomes such as phantom reads.

Summary

- Four types of NoSQL Databases are 1) Key-value 2)Column-oriented 3) Document-oriented 4) Graphs
- NOSQL can handle structured, semi-structured, and unstructured data with equal effect
- CAP theorem consists of three words Consistency, Availability, and Partition Tolerance
- BASE stands for **B**asically **A**vailable, **S**oft state, **E**ventual consistency
- The term "eventual consistency" means to have copies of data on multiple machines to get high availability and scalability
- NOSQL offer limited query capabilities
- Principal Features of NoSQL.


HEY, UH, COULD
YOU GET ME A TABLE
mongo DB
Kalash
JSON
NO TABLE,
ONLY DOCUMENT
imgflip.com

